INTERNAL PRODUCT DOCUMENTATION

Workforce Portal

Employee Lifecycle, Timesheets & Payroll — a two-sided platform for Employers/Admins and Employees, built on a NestJS microservice backend and two Angular applications.

Employer Portal | Employee Portal | NestJS Microservices | Angular 21 + NgRx | MongoDB



Version 1.0 · Documentation Release

July 2026

Table of Contents

1 **Project Overview** — What the system is and who it's for

2 **Scope of the Project** — In-scope modules, out-of-scope items, core principles

3 **User Roles & Permissions** — Employer/Admin vs. Employee capabilities

4 **System Architecture** — Portals, microservices, database, tech stack

5 **Getting Started** — Local setup and run instructions

6 **Employer / Admin Portal — Features** — Every screen and capability

7 **Employee Portal — Features** — Every screen and capability

8 **Key Workflows & Flow Charts** — Onboarding, login, timesheet approval, payroll run

9 **Backend Services Reference** — All six services, ports, responsibilities

10 **Data Model Summary** — Core entities at a glance

11 **Current Build Status** — What's fully built vs. in progress

12 **Scope of New / Upcoming Features** — Roadmap and known gaps

13 **Appendix** — Repository layout & environment variables

How to use this document — Sections 1–3 give the product picture (what it does and for whom). Sections 4–5 are for anyone setting the system up. Sections 6–8 are the feature/workflow reference. Sections 9–13 are the technical appendix for engineers extending the system.

1. Project Overview

Workforce Portal is a two-sided system for running a company's employee lifecycle, timesheets, and payroll end to end. It replaces manual, spreadsheet-driven HR processes with a single source of truth: the employer creates every employee record, employees log in to submit their own hours, and every approval, payroll run, and status change is tracked with a full audit trail.

- **Employer / Admin Portal**

  Used by HR and company admins to create and manage employee accounts, review and approve timesheets with remarks, run payroll for a period, and (in the upcoming release) respond to employee feedback.

- **Employee Portal**

  Used by staff to log in, complete first-time password setup, submit timesheets for approval, and — once the next release ships — view payslip history and raise feedback/concerns directly in the portal.

1.1 Why it exists

The project was built from an original functional specification (`Employer-Employee-Portal-Documentation.pdf`) describing the full narrative of every screen. This document reflects how that specification was actually implemented — the working architecture, data model, and every screen and form field in the two live applications.

1.2 Who this document is for

Audience	What they should read
Product / Business stakeholders	Sections 1–3 and 6–8 for scope, roles, and feature walkthroughs
New engineers joining the project	Sections 4–5 and 9–11 for architecture, setup, and data model
QA / Testers	Sections 6–8 for exact form fields, validations, and workflow states
Project planning / delivery	Sections 11–12 for what's shipped vs. what's next

1.3 At a glance

Aspect	Detail
Applications	2 Angular front-ends (Employer Portal, Employee Portal) + 6 NestJS backend services
Database	Single shared MongoDB instance, logically separated by service ownership
Authentication	Email + password login issuing JWT access & refresh tokens; email-OTP gated password reset
Core modules	Auth, Employees, Timesheets, Payroll, Feedback, Notifications
Deployment shape	No API gateway, no event bus — direct service-to-service HTTP calls (see §4)

2. Scope of the Project

2.1 In scope

- Employer-managed employee directory — create, edit, activate/deactivate (soft-delete only)
- Secure authentication for both roles: password login, JWT session, refresh-token renewal, OTP-gated password reset
- Timesheet lifecycle — draft, submit, approve/reject with remarks, full status-history audit trail
- Payroll runs — per-period pay computation from approved hours and employee pay rates, processing, payslip notification
- Feedback loop — employees raise issues by category, employers reply and change status
- Automated email notifications for every account, timesheet, payroll, and feedback event

2.2 Out of scope (current release)

- Employee self-registration — accounts are always employer-provisioned
- Leave management, attendance/biometric integration
- Partial-period or pro-rated payroll (leave-adjusted pay)
- Payroll deduction rules (tax, benefits) — currently always zero
- Manager-routed (multi-level) timesheet approval — all approvals route to any company admin
- Multi-company / multi-tenant admin console beyond per-company data scoping

2.3 Core operating principles

Employer is the source of truth

Every employee account is created by an employer/admin. The system auto-provisions login credentials; the employee never self-registers.

Single-step login for both roles

Password verification alone issues tokens for both Employee and Employer/Admin accounts — no OTP at login. OTP exists only for password recovery.

Nothing is hard-deleted

Employees, timesheets, feedback, and payroll runs all carry a status/visibility flag instead of being removed — full history is always preserved for audit.

Every query is scoped

`companyId` and `employeeId` are read from the verified JWT — never trusted from a request body — and applied to every query.

Known scope gap — deactivating an employee (`isActive:false`) currently hides them from directory views and payroll, but the login service does not yet cross-check that flag before issuing tokens. See §12 for the planned fix.

3. User Roles & Permissions

The system recognizes exactly two roles. Role is embedded in the signed JWT (`role` claim) and enforced on every route by a shared guard — a role can never be escalated client-side.

3.1 EMPLOYER_ADMIN — Employer / Admin

Capability	Detail
Employee management	Create employee records, edit details, activate/deactivate access
Timesheet review	View the company-wide approval queue; approve, reject (remarks required), or keep pending
Payroll	Trigger a payroll run for a period, process runs, view all pay history
Feedback	View the company feedback inbox, reply, change status **screen in progress**
Visibility	Company-wide data only — never another company's records

3.2 EMPLOYEE — Staff

Capability	Detail
Timesheets	Log daily hours, save as draft, submit for approval, view own status/remarks history
Profile	View own contact/employment details (read-only), change password via emailed verification code
Payroll	View own payslip history **screen in progress**
Feedback	Raise a feedback item by category, track its status **screen in progress**
Visibility	Own records only — every request is force-scoped to the caller's own `employeeId`

3.3 Role enforcement, front to back

1. Each Angular app rejects a login client-side if the returned role doesn't match that portal (an employee cannot log into the Employer Portal, and vice versa).
2. Every backend route is annotated with an allowed-roles list; a shared `RolesGuard` throws a 403 if the caller's JWT role isn't on the list.
3. Every controller reads `companyId` / `employeeId` from the verified token and applies it as a hard query filter — an Employee calling another employee's payroll endpoint is silently redirected to their own record server-side.

4. System Architecture

Two independent Angular applications talk directly to six independent NestJS backend services — there is no API gateway and no event bus. Cross-service calls are plain, synchronous HTTP.

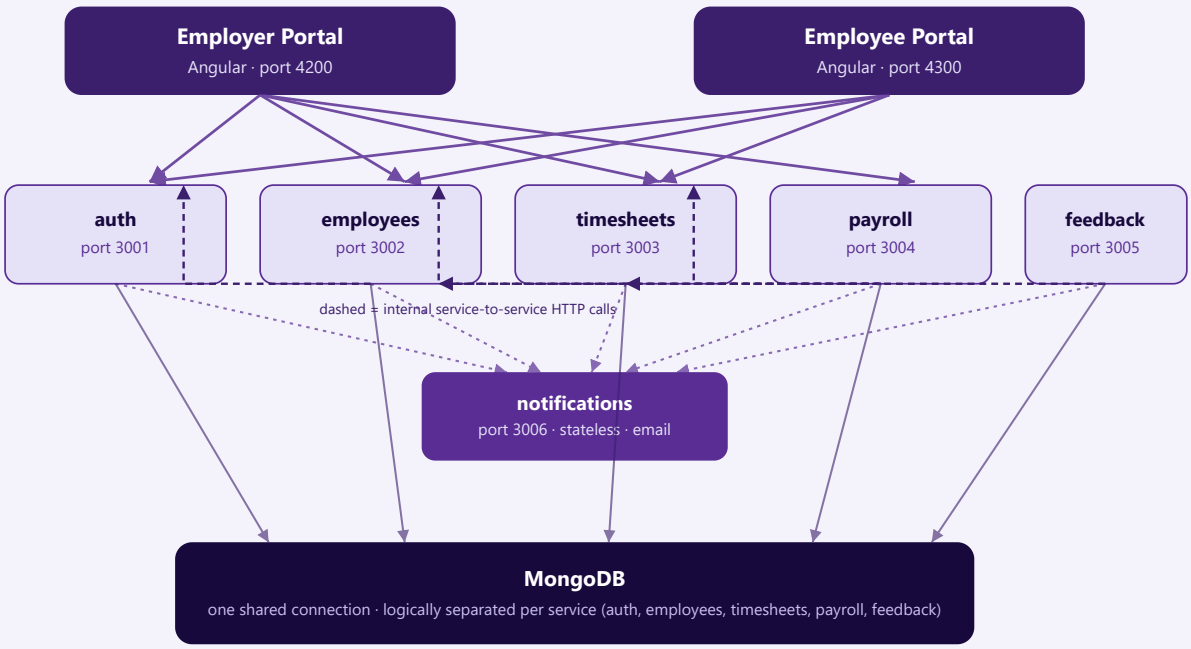


Fig. 4.1 — Both portals call every backend service directly on its own port; solid arrows are browser calls, dashed arrows are internal service-to-service calls, all services share one MongoDB instance except the stateless `notifications` service.

4.1 Why no gateway and no event bus

The original specification diagrammed an API Gateway in front of all services plus an event bus for cross-service triggers. This build deliberately simplifies that: each frontend calls each backend service directly, and the handful of cross-service calls needed (provisioning a login, resolving employee pay details, pulling approved hours, sending email) are typed HTTP helper functions in a shared library. This is a conscious trade-off — a cross-service call fails synchronously and visibly rather than being silently retried by a broker — judged simpler to build and operate at this system's size.

4.2 Technology stack

Layer	Technology
Frontend framework	Angular 21, standalone components, signals, `@if` / `@for` control flow
Frontend state	NgRx (store + effects) for server-derived state; component signals for local UI state
Frontend styling	Bootstrap 5 + Tailwind 4, driven by a shared CSS custom-property design-token set
Backend framework	NestJS, Nest CLI monorepo (six independent applications, one shared library)
Backend pattern	Controller → Service → Repository → Schema, per feature module
Database	MongoDB via Mongoose

Layer	Technology
Auth	JWT (access + refresh), Passport strategy, bcrypt password hashing
Email	SMTP via the `notifications` service; logs to console when unconfigured

5. Getting Started

The system runs entirely locally with Docker (for MongoDB only) and plain `npm` commands for every application — no gateway or broker to stand up.

5.1 Prerequisites

- Node.js and npm
- Docker (for the local MongoDB container)

5.2 Run sequence

```
docker compose up -d mongo

cd backend
npm install
npm run start:all
# boots all six services together (auth, employees, timesheets,
# payroll, feedback, notifications) with one colored log stream each

npm run seed:employer -- --name "Jane Doe" --email jane@company.com --password Str0ngPassword!
# creates the first employer/admin account

cd ../employer-portal
npm install
npm start   # http://localhost:4200

cd ../employee-portal
npm install
npm start   # http://localhost:4300
```

5.3 First login

1. Open `http://localhost:4200` and log in with the seeded employer/admin credentials.
2. Create an employee from the Employees screen — their login is provisioned automatically and a temporary password is emailed (or logged to console if SMTP isn't configured).
3. Open `http://localhost:4300` and log in with the new employee's credentials to explore the employee-side experience.

> **No SMTP required to try it end to end —** if `MAIL_HOST` is left blank in `backend/.env`, every email (temporary passwords, OTP codes, status notifications) is printed to the service's console log instead of sent, so the whole system is runnable without configuring real email.

6. Employer / Admin Portal — Features

6.1 Login

Email + password. A valid password alone issues tokens — no OTP step. If the account's role isn't `employer_admin`, the portal shows an error directing the user to the Employee Portal instead.

6.2 Dashboard

Read-only KPI overview: total employees, active count, inactive count, and timesheets pending review, plus a short list of pending timesheets and quick-action links into Employees, Timesheets, and Payroll.

6.3 Employees

A searchable/filterable directory (name, email, department, joined date, active/inactive status, an Activate/Deactivate toggle) with a collapsible create-employee form above it.

Field	Type	Required
Full name	Text	Yes, min 2 characters
Official email	Email	Yes — becomes the login ID
Department	Text	No
Designation	Text	No
Joining date	Date picker	Yes
Salary / rate	Number	Yes, ≥ 0
Pay type	Select — Fixed / Hourly	Defaults to Fixed
Initial password	Text	No — auto-generated if left blank

Deactivating/reactivating an employee is a single reversible toggle — no confirmation dialog, since the underlying record and history are never deleted.

6.4 Timesheets — approval queue

One card per submitted timesheet: period, total hours, current status pill, and the full remarks/status-history thread if one exists. Any timesheet still `pending` shows a decision panel:

Action	Remarks	Result
Approve	Optional	Status → `approved`, employee notified by email
Reject	**Required**	Status → rejected, employee can edit and resubmit
Keep Pending	Optional	Stays in the queue, remark still recorded

6.5 Payroll & Feedback Inbox

> Both screens currently show a placeholder panel — the backend APIs are fully built and can be exercised via Swagger today, but the Employer Portal UI for these two screens is scheduled for the next release. See §12.

7. Employee Portal — Features

7.1 Login & password recovery

Same email + password login as the Employer Portal (no OTP step). A **Forgot password?** link starts a separate two-step recovery flow:

Screen	Field(s)	Validation
Forgot password	Email	Required, valid email
Reset password	6-digit code, New password, Confirm password	Code exactly 6 digits; passwords min 8 chars and must match

The reset screen includes a **Resend code** control. This same flow is reused for changing a password while logged in — there's no separate authenticated change-password form.

7.2 Dashboard

Read-only pending/approved/rejected timesheet counts and quick-action links. If `mustChangePassword` is still true (a fresh account on its first login), a banner links to Profile until dismissed by navigation.

7.3 Timesheets — log hours

A collapsible log-hours form with a repeatable entry row, plus a history list below it.

Field	Type	Validation
Period	Text	Required, e.g. `2026-07-14`
Entry — Date	Date picker	Per row
Entry — Hours	Number	Per row, ≥ 0
Entry — Task	Text	Optional, per row

Rows can be added (**+ Add day**) or removed. **Save as draft** keeps the timesheet editable; **Submit for review** moves it into the employer's approval queue. Any `rejected` timesheet can be edited and resubmitted the same way.

7.4 Profile

Read-only contact info (name, email, department, designation) plus a **Change password** link that routes through the same emailed verification-code flow as password recovery (§7.1).

7.5 Payroll, Feedback & My Insights

> All three screens currently show a placeholder panel — backend APIs are fully built (view payslip history, raise/track feedback), the Employee Portal UI for these is scheduled for the next release. See §12.

8. Key Workflows & Flow Charts

8.1 Employee onboarding & account provisioning



Fig. 8.1 — One form submission creates both the employee record and its login in a single flow; the employee never self-registers.

8.2 Login & password recovery

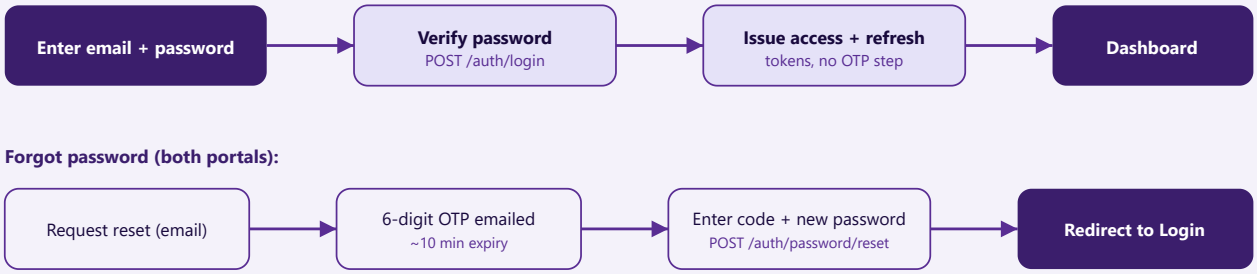


Fig. 8.2 — Login is always a single step for both roles; the OTP mechanism exists solely for the forgot/reset-password flow.

8.3 Timesheet submission & approval

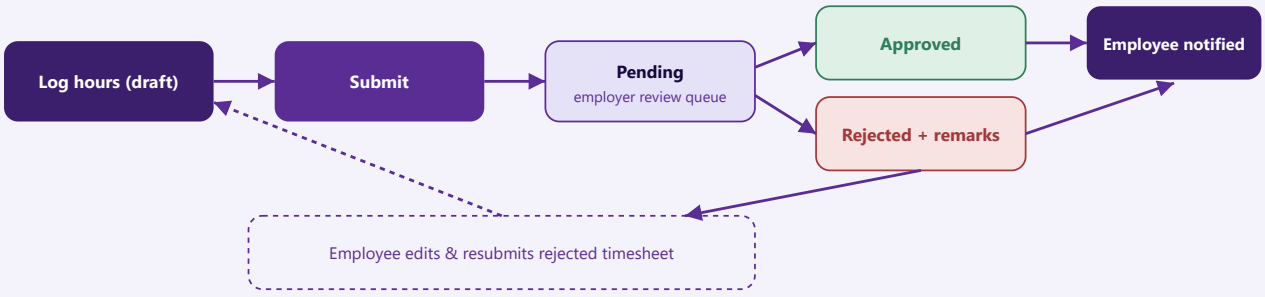


Fig. 8.3 — Every decision appends to an audit trail (`statusHistory`); a rejection always carries a remark and routes back to the employee for resubmission.

8.4 Payroll run

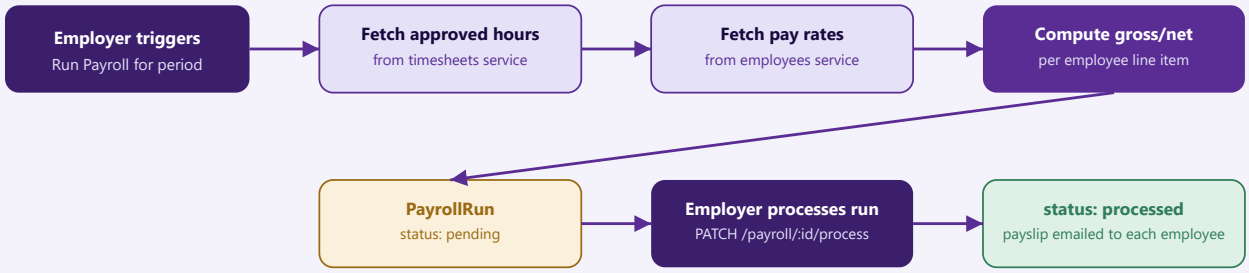


Fig. 8.4 — A payroll run is computed fresh from approved timesheet hours and current pay rates; corrections create a new run and void the old one rather than editing history in place.

8.5 Feedback lifecycle

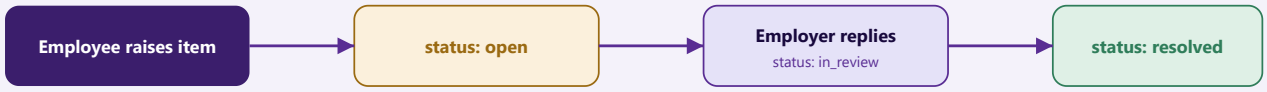


Fig. 8.5 — Each reply appends to a thread and emails the employee; resolved items move to an archived view rather than being deleted.

9. Backend Services Reference

Service	Port	Owns	Responsibility
auth	3001	`User`	Login, JWT issuance/refresh, password reset OTP, account provisioning (called internally by `employees`)
employees	3002	`Employee`	Employee directory, create/edit, activate/deactivate, resolves employee details for other services
timesheets	3003	`Timesheet`	Draft/submit/approve/reject cycle, status-history audit trail, exposes approved hours to `payroll`
payroll	3004	`PayrollRun`	Per-period pay computation, run processing, payslip notification
feedback	3005	`Feedback`	Employee-raised issues, reply thread, status changes
notifications	3006	— (stateless)	Sends every system email; no database connection, no browser calls it directly

9.1 Cross-service calls

Caller → Callee	Purpose
employees → auth	Provision the login credential when an employee is created
auth / employees / timesheets / payroll / feedback → notifications	Trigger the relevant transactional email
timesheets / payroll / feedback → employees	Resolve an employee's name / email / pay details
payroll → timesheets	Pull approved hours for a period when a run is triggered

10. Data Model Summary

10.1 User (auth)

`name`, `email` (login ID), `passwordHash`, `role`, `companyId`, `employeeId`, `mustChangePassword`, `otpHash` / `otpExpiresAt`, hashed refresh token.

10.2 Employee (employees)

`companyId`, `name`, `email`, `department`, `designation`, `joiningDate`, `salary`, `payType` (hourly/fixed), `reportingManagerId`, `isActive`.

10.3 Timesheet (timesheets)

`employeeId`, `companyId`, `period`, `entries[]` (date, hours, task), `status` (draft/pending/approved/rejected), append-only `statusHistory[]`, `isWithdrawn`.

10.4 PayrollRun (payroll)

`companyId`, `period`, `lineItems[]` (employeeId, hoursApproved, grossPay, deductions, netPay), `status` (pending/processed/paid), `isVoid`.

10.5 Feedback (feedback)

`employeeId`, `companyId`, `category` (payroll/timesheet/portal_bug/general), `description`, `status` (open/in_review/resolved), append-only `replies[]`, `isArchived`.

10.6 Soft-delete flags, at a glance

Entity	Flag(s)	Effect
Employee	isActive	Hides from active directory views; history preserved
Timesheet	status, isWithdrawn	Drops from active list, stays queryable for audit
Feedback	status, isArchived	Moves to archived view instead of disappearing
PayrollRun	isVoid	Corrections create a new run; the old one is voided, not edited

11. Current Build Status

The foundation — authentication, the employee directory, and the full timesheet lifecycle — is solid on both the backend and both frontends. Payroll and feedback are fully built and testable on the backend; their frontend screens are the next scheduled release.

Area	Backend	Frontend
Auth (login, OTP, JWT, password reset)	Full	Full
Employees (directory, create, status)	Full	Full (employer side)
Timesheets (draft, submit, approve, remarks)	Full	Full (both sides)
Payroll (runs, processing, payslip history)	Full	Placeholder screen
Feedback (raise, reply, status)	Full	Placeholder screen
Analytics ("My Insights")	—	Placeholder screen

> This was a deliberate scope decision, not an oversight: keep the foundation (auth, employees, timesheets) solid rather than spreading thin effort across every module at once. The payroll and feedback backend APIs are real and ready to build a UI against today.

12. Scope of New / Upcoming Features

12.1 Next release — frontend build-out

Payroll screens (both portals)

Employer: trigger/process runs, view all history. Employee: view own payslip history and per-run breakdown. Backend already supports both end to end.

Feedback screens (both portals)

Employer: company-wide inbox with reply/status actions. Employee: raise new items by category and track their thread. Backend already supports both end to end.

Employee "My Insights" analytics

A personal analytics view (hours trends, approval rate, pay history charts) — no backend endpoint yet; needs design + API work.

isActive login enforcement

Denormalize `isActive` onto the `auth` User record (or cross-check it at login) so a deactivated employee is actually blocked from authenticating, closing the current documented-vs-implemented gap.

12.2 Further-out candidates

- **Payroll deduction rules** — tax/benefit deduction support; `deductions` is currently always zero.
- **Leave-adjusted / pro-rated payroll** — partial-period pay for employees who joined, left, or took leave mid-period.
- **Manager-routed timesheet approval** — route submissions to an employee's `reportingManagerId` instead of any company admin; the field already exists on the Employee schema but isn't used for routing yet.
- **Employer-side feedback analytics** — trend/volume reporting over the feedback inbox once the base screen ships.

> **Scope discipline —** per the project's core principles (§2.3), new features should continue the existing patterns: soft-delete flags over hard deletes, JWT-scoped queries, and direct service-to-service HTTP calls rather than introducing a gateway or broker for a system at this scale.

13. Appendix

13.1 Repository layout

```
workforce-portal/
  backend/                Nest CLI monorepo
    apps/
      auth/                port 3001
      employees/           port 3002
      timesheets/          port 3003
      payroll/             port 3004
      feedback/             port 3005
      notifications/        port 3006
    libs/shared/           cross-cutting Nest code (guards, JWT, enums)
    scripts/seed-employer.ts
  employer-portal/         Angular app, port 4200
  employee-portal/         Angular app, port 4300
  Docs/                    project documentation
  docker-compose.yml        local MongoDB for dev
  README.md                 quick-start run instructions
```

13.2 Key environment variables (backend/.env)

Variable	Used by	Purpose
APP_MONGODB_URI	All except notifications	Shared Mongo connection string
JWT_SECRET / JWT_ACCESS_EXPIRY	All	Access token signing/verification
JWT_REFRESH_SECRET / JWT_REFRESH_EXPIRY	auth	Refresh token signing
<APP>_PORT	Each service	Which port it listens on
MAIL_HOST / MAIL_PORT / MAIL_USER / MAIL_PASS / MAIL_FROM	notifications	SMTP config — blank = console logging only
EMPLOYER_PORTAL_URL / EMPLOYEE_PORTAL_URL	auth	Link targets used inside emails
*_SERVICE_URL (per service)	Calling service	Base URLs for cross-service HTTP calls

13.3 Document source

This documentation was generated from the project's internal architecture notes (`Docs/project-overview.md`, `Docs/backend-architecture.md`, `Docs/frontend-architecture.md`) and a direct review of the `backend/`, `employer-portal/`, and `employee-portal/` source trees.

> End of document — Workforce Portal Documentation, v1.0.